

Table of Contents

Stage 1: Audit and Compatibility Check	3
1.1 Vercel PHP Runtime: The Reality	3
1.2 Incompatible Features Checklist	3
1.3 When to Choose an Alternative Platform	4
Stage 2: GitHub Preparation	5
2.1 Initialize Git Locally	5
2.2 Create .gitignore File	6
2.3 Remove XAMPP-Specific Files	7
2.4 First Commit and Push to GitHub	7
2.5 Recommended Repository Structure	8
Stage 3: Migrating MySQL to Turso	8
3.1 Understanding Turso Architecture	8
3.2 MySQL vs LibSQL: Key Differences	9
3.3 Install Turso CLI and Create Database	10
3.4 Schema and Data Migration Strategy	10
Stage 4: Refactoring PHP Database Layer	11
4.1 Install Turso PHP SDK	11
4.2 Before and After: Database Connection	12
4.3 Before and After: Prepared Statements	12
4.4 Environment-Aware Configuration File	12
4.5 Common Query Pattern Migrations	13
Stage 5: Preparing PHP for Vercel	14
5.1 How Vercel Handles PHP	14
5.2 Required vercel.json Configuration	14

5.3 Folder Structure for Vercel	15
5.4 Routing Adjustments	16
Stage 6: Deploying to Vercel	16
6.1 Vercel Dashboard Deployment Steps	16
6.2 Environment Variable Setup	16
6.3 Custom Domain Setup	17
Stage 7: Handling Hardcoded Localhost URLs and Paths	17
7.1 Dynamic Base URL Configuration	18
7.2 Before and After: Common Hardcoded Patterns	18
7.3 Bulk Search and Replace Script	19
Stage 8: SSL, Environment Security, and Production Hardening	19
8.1 HTTPS and SSL	19
8.2 Secure Headers	20
8.3 Hiding PHP Errors in Production	20
8.4 Secrets Management	21
Stage 9: Post-Deployment Testing	21
9.1 Testing Checklist	21
9.2 Database Connectivity Check	22
9.3 Debugging Strategy	22
9.4 Performance Checks	23
Stage 10: Troubleshooting Reference	23
Appendix A: Recommended Architecture for Long-Term Scalability	24
Appendix B: Platform Recommendation	25
Appendix C: Migration Difficulty Estimate	25

Stage 1: Audit and Compatibility Check

Before writing a single line of code or pushing any file to GitHub, you must conduct a thorough audit of your existing PHP/MySQL project. This stage determines the feasibility of running your application on Vercel, identifies incompatible features that need refactoring, and helps you decide whether Vercel is even the right platform for your workload. Skipping this audit is the single most common cause of failed migrations.

The audit covers three dimensions: PHP feature compatibility with Vercel's execution model, database migration complexity from MySQL to Turso (LibSQL), and architectural fit between your application's patterns and Vercel's serverless paradigm. Each dimension has specific checkpoints that will inform the refactoring effort required in subsequent stages.

1.1 Vercel PHP Runtime: The Reality

Vercel does not have an officially supported PHP runtime. Vercel's native runtimes are limited to Node.js, Go, Python, and Ruby. However, PHP can run on Vercel through two mechanisms. The first is the community-maintained PHP runtime via the [@vercel/php](#) builder, which uses a custom Buildpack under the hood. This builder compiles PHP from source and creates serverless functions from your PHP files. The second approach involves converting PHP files into serverless API routes where each `.php` file becomes a serverless function that handles HTTP requests.

The community runtime supports PHP 8.x and common extensions like `pdo_mysql`, `mysqli`, `mbstring`, `openssl`, `curl`, and `gd`. However, it has significant limitations that you must understand before proceeding. Each PHP file is deployed as an independent serverless function with its own cold-start time (typically 200-500ms for PHP). There is no persistent filesystem between requests: any file written during one invocation is gone in the next. There are strict execution timeouts (10 seconds for hobby, 60 seconds for pro plans). Background processes, cron jobs, and long-running tasks are not supported.

1.2 Incompatible Features Checklist

Review your codebase against this checklist. Any item marked "Incompatible" will require refactoring or an alternative approach. The severity column indicates how difficult the workaround will be.

Feature	Vercel Compatible?	Workaround	Severity
<code>mysql_*</code> functions (deprecated)	No	Replace with PDO or Turso SDK	Medium
PDO/MySQLi persistent connections	No	Connection per request (stateless)	Low

Feature	Vercel Compatible?	Workaround	Severity
Local file writes (uploads, logs)	No	Use S3/Upstash/object storage	High
PHP sessions (file-based)	No	Use KV store (Upstash Redis)	Medium
Cron jobs / scheduled tasks	No	External scheduler (cron-job.org)	Medium
\$_SERVER["DOCUMENT_ROOT"]	Limited	Use __DIR__ or env vars	Low
.htaccess / Apache config	No	Use vercel.json rewrites	Medium
Header() redirects	Yes	Works normally in serverless	None
JSON API endpoints	Yes	Works perfectly as serverless	None
Static file serving	Yes	Place in /public directory	None

Table 1: PHP Feature Compatibility Matrix for Vercel

1.3 When to Choose an Alternative Platform

Vercel is ideal for static sites, serverless APIs, and frontend-heavy applications. However, if your PHP application relies heavily on any of the following, you should seriously consider alternative hosting platforms. Traditional PHP apps with heavy session usage (e-commerce carts, user authentication flows that depend on server-side sessions) will require significant refactoring. Applications that process large file uploads (image galleries, document management systems) cannot use Vercel's ephemeral filesystem. Projects with background workers (email queues, report generation, data processing) need a persistent runtime environment.

For these scenarios, consider the following alternatives. **Railway** offers native PHP support with persistent filesystem, proper cron jobs, and MySQL/PostgreSQL databases starting at \$5/month. **Render** provides PHP web services with persistent disks, background workers, and free SSL certificates. **Hostinger** or traditional shared hosting remains the simplest option if your application is a standard PHP website that does not need the edge deployment benefits of Vercel. The decision matrix below can help you choose.

Criterion	Vercel + Turso	Railway	Render	Traditional Hosting
PHP support	Community runtime	Native	Native	Native

Criterion	Vercel + Turso	Railway	Render	Traditional Hosting
Persistent filesystem	No	Yes	Yes (disk)	Yes
MySQL/MariaDB	No (Turso only)	Yes (managed)	Yes (managed)	Yes (included)
Session handling	Requires external KV	Native	Native	Native
Cron jobs	No	Yes	Yes (cron)	Yes (cPanel)
Cold start latency	200-500ms	None (persistent)	None (persistent)	None
Edge deployment	Yes (global)	No	No	No
Cost (starter)	Free tier available	\$5/month	Free tier available	\$2-5/month
Best for	APIs, static + API	Full-stack PHP	Full-stack PHP	Traditional PHP sites

Table 2: Platform Comparison for PHP Applications

WARNING: If your application has more than 3 incompatible features from Table 1, strongly consider Railway or Render instead of Vercel. The refactoring cost may exceed the benefits of Vercel's edge deployment.

Stage 2: GitHub Preparation

With the audit complete, the next step is to initialize Git version control for your project and push it to GitHub. This is not just a deployment prerequisite; it is a fundamental best practice for any production project. GitHub provides version history, collaboration capabilities, CI/CD integration via GitHub Actions, and serves as the source of truth that Vercel connects to for automatic deployments.

2.1 Initialize Git Locally

Navigate to your project root directory (the top-level folder of your XAMPP htdocs project) and run the following commands. These commands initialize a new Git repository, configure your identity (required for commits), and prepare the project for its first commit. Replace the name and email with your actual GitHub credentials.

```
# Navigate to your project root (e.g., C:/xampp/htdocs/myproject)
cd /path/to/your/project

# Initialize Git repository
```

```
git init

# Configure your identity (one-time setup)
git config user.name "Your Name"
git config user.email "your.email@example.com"
```

2.2 Create .gitignore File

The .gitignore file is critical. Without it, you will accidentally commit XAMPP-specific files, database credentials, and temporary files to your public repository. The following .gitignore covers the most common files that should never be tracked in version control. Create this file in your project root.

```
# === XAMPP / Local Environment ===
xampp/
htdocs/
.htaccess.bak

# === PHP ===
vendor/
composer.lock
*.cache

# === Environment & Secrets ===
.env
.env.local
.env.production
.env.*.local
config.php
database_credentials.php

# === Uploads & Generated Files ===
uploads/
tmp/
logs/
cache/

# === OS Files ===
.DS_Store
Thumbs.db
desktop.ini

# === IDE ===
.vscode/
.idea/
*.swp
```

```
*.SWO

# === Node (if using Vercel CLI) ===
node_modules/
.vercel/
```

2.3 Remove XAMPP-Specific Files

Before making your first commit, remove any files that reference localhost URLs, absolute XAMPP paths, or contain database credentials stored in plain text. Common patterns include configuration files with hardcoded connection strings, PHP files with paths like `C:/xampp/htdocs/...`, and any backup files generated by XAMPP or your editor. Use the following commands to identify and remove these references.

```
# Find files with localhost references
# (Review output carefully before deleting)
grep -rl "localhost" --include="*.php" .
grep -rl "127.0.0.1" --include="*.php" .
grep -rl "C:/xampp" --include="*.php" .

# Remove XAMPP-specific files if present
rm -f .htaccess.bak
rm -rf xampp_backup/
```

2.4 First Commit and Push to GitHub

After creating the repository on GitHub (at github.com/new), push your local code. Create the repository as Private if your project contains any sensitive business logic. Use the following commands to stage all files, create the initial commit, and push to the remote repository.

```
# Stage all files
git add .

# Create initial commit
git commit -m "feat: initial commit - PHP project structure"

# Add remote origin (replace with your repo URL)
git remote add origin https://github.com/[YOUR_GITHUB_REPO].git

# Push to main branch
git branch -M main
git push -u origin main
```

NOTE: If your project has a `config.php` or `db.php` with hardcoded credentials, create a `config.example.php` with placeholder values and commit that instead. Add the real config file to `.gitignore`. This is covered in detail in Stage 7.

2.5 Recommended Repository Structure

Organize your project following this structure before pushing to GitHub. This layout is compatible with both Vercel's deployment expectations and standard PHP project conventions. The key principle is that all publicly accessible files (CSS, JavaScript, images) go in the `public/` directory, while PHP logic files remain at the root or in an `api/` directory.

myproject/
-- public/ # Static assets served by Vercel
-- css/
-- js/
-- images/
-- index.php # Entry point
-- api/ # Serverless API endpoints
-- users.php
-- products.php
-- auth.php
-- config/
-- database.php # DB connection (reads from env)
-- app.php # App-level config
-- includes/ # Shared PHP files
-- functions.php
-- headers.php
-- auth.php
-- .gitignore
-- .env.example # Template for environment variables
-- vercel.json # Vercel configuration
-- composer.json # PHP dependencies

Stage 3: Migrating MySQL to Turso

Turso is a distributed SQLite-compatible database built on libSQL, an open-source fork of SQLite. It provides edge-deployed database replicas with automatic synchronization, which makes it an excellent choice for serverless applications. However, migrating from MySQL requires understanding the fundamental differences between the two database systems, as they are not fully compatible despite both supporting SQL.

3.1 Understanding Turso Architecture

Turso operates on a unique architecture that differs significantly from traditional MySQL databases. The core database engine is libSQL, which is SQLite-compatible with additional features like remote

authentication, network replication, and built-in branching. When you create a Turso database, it runs on Turso's infrastructure with the primary replica in a specific region (e.g., eu-west-1 or us-east-1). You can create additional read-only replicas in other regions for low-latency reads.

Connections are made over HTTPS using a URL format: `libsql://[YOUR_DB_NAME]-[YOUR_ORG].turso.io`. Authentication is handled via an auth token that you generate from the Turso CLI or dashboard. All queries are sent over HTTP, which means there is no persistent TCP connection like MySQL. This fits naturally with Vercel's serverless model where each function invocation is independent.

3.2 MySQL vs LibSQL: Key Differences

While both databases support SQL, there are critical syntax and behavioral differences that will affect your migration. Understanding these differences before writing any migration scripts will save you significant debugging time. The most impactful differences relate to data types, auto-increment behavior, and SQL function compatibility.

Feature	MySQL	Turso (LibSQL)	Migration Action
Auto-increment	AUTO_INCREMENT	INTEGER PRIMARY KEY AUTOINCREMENT	Rewrite CREATE TABLE
Boolean type	BOOLEAN (TINYINT(1))	INTEGER (0/1)	No change needed
DATETIME	DATETIME, TIMESTAMP	TEXT (ISO 8601 string)	Use strftime() or store TEXT
ENUM	ENUM("a","b")	TEXT with CHECK constraint	Rewrite as TEXT + CHECK
UNSIGNED	UNSIGNED INT	Not supported	Remove UNSIGNED keyword
LIMIT offset	LIMIT 10 OFFSET 5	LIMIT 5, 10 (comma syntax)	Swap order
CONCAT	CONCAT(a, b)	a b (pipe operator)	Replace function calls
NOW()	NOW()	datetime("now")	Replace function calls
IF()	IF(cond, a, b)	CASE WHEN cond THEN a ELSE b END	Rewrite conditionals
GROUP_CONCAT	GROUP_CONCAT(col)	GROUP_CONCAT(col, separator)	Add separator param
Foreign keys	Enabled by default	Must PRAGMA enable	Add PRAGMA statement

Feature	MySQL	Turso (LibSQL)	Migration Action
JSON	JSON column type	JSON1 extension (built-in)	Test compatibility

Table 3: MySQL to LibSQL Syntax Differences

3.3 Install Turso CLI and Create Database

The Turso CLI is the primary tool for managing your databases, replicas, and authentication tokens. Install it using the commands below for your operating system. After installation, authenticate with your Turso account (create one at turbo.so if you have not already), then create your database and generate the auth token needed for connections.

Install Turso CLI (macOS / Linux)
<code>curl -sSfL https://get.turso.so/install.sh bash</code>
Install Turso CLI (Windows via PowerShell)
<code>irm https://get.turso.so/install.ps1 iex</code>
Authenticate with Turso
<code>turso auth login</code>
Create a new database
<code>turso db create [YOUR_DB_NAME] --region eu-west-1</code>
Get the database URL
<code>turso db show [YOUR_DB_NAME] --url</code>
Output: <code>libsql://[YOUR_DB_NAME]-[YOUR_ORG].turso.io</code>
Generate an auth token
<code>turso db tokens create [YOUR_DB_NAME]</code>
Output: a long token string - SAVE THIS SECURELY
Create a replica in another region (optional)
<code>turso db replicas create [YOUR_DB_NAME] --region us-east-1</code>

3.4 Schema and Data Migration Strategy

The recommended approach for migrating your MySQL schema and data to Turso is a three-step process: first, export your MySQL schema; second, convert the syntax to SQLite-compatible DDL; third, export and import the data. For small to medium databases (under 100MB), you can use `mysqldump` for export and a conversion script for import. For larger databases, consider using a dedicated migration tool or writing a PHP script that reads from MySQL and writes to Turso.

Start by exporting your MySQL schema and data. Then convert the MySQL DDL to SQLite-compatible syntax using the mapping table above. The most common conversion is replacing `AUTO_INCREMENT` with `AUTOINCREMENT` and removing `UNSIGNED` keywords. After conversion, use the Turso CLI to import the

schema directly into your database.

```
# Step 1: Export MySQL schema (no data, structure only)
mysqldump -u root --no-data mydatabase > schema.sql

# Step 2: Export MySQL data (CSV format)
mysqldump -u root --tab=/tmp/export --fields-terminated-by="," \
--fields-enclosed-by="'"' --lines-terminated-by="\n" mydatabase

# Step 3: Convert schema.sql (manual edit required)
# - Replace AUTO_INCREMENT with AUTOINCREMENT
# - Remove UNSIGNED keywords
# - Replace ENUM with TEXT
# - Replace DATETIME with TEXT
# - Remove ENGINE=InnoDB DEFAULT CHARSET lines
# - Add: PRAGMA foreign_keys = ON; at the top

# Step 4: Import schema into Turso
turso db shell [YOUR_DB_NAME] < schema_converted.sql

# Step 5: Import data (use turso db shell with .import)
turso db shell [YOUR_DB_NAME]
.mode csv
.import /tmp/export/users.csv users
.import /tmp/export/products.csv products
```

WARNING: LibSQL does not support MySQL-specific features like stored procedures, triggers (with MySQL syntax), views (partially), and full-text search with MySQL syntax. If your application relies on these features, they must be reimplemented in application code or using LibSQL-compatible alternatives.

Stage 4: Refactoring PHP Database Layer

This is the most code-intensive stage of the migration. Every database interaction in your PHP application must be rewritten from MySQL PDO/MySQLi to use the Turso PHP SDK (libsql-client). This involves replacing the connection logic, rewriting prepared statements, and ensuring all queries use SQLite-compatible syntax. The good news is that the Turso PHP SDK provides a clean, modern API that is actually simpler than PDO for most use cases.

4.1 Install Turso PHP SDK

The official Turso PHP SDK (libsql-php) provides a native PHP client for connecting to Turso databases over HTTP. It supports both local SQLite files and remote Turso databases. Install it via Composer in your project root. If you do not have Composer installed, download it from getcomposer.org.

```
# Install Turso PHP SDK via Composer
composer require turso/libsql-client
```

4.2 Before and After: Database Connection

The following comparison shows a typical MySQL PDO connection and its Turso equivalent. The key differences are: the Turso SDK uses a URL and auth token instead of host/port/username/password, the connection is established over HTTP (no persistent TCP connection), and the client object is used directly for queries instead of a PDO instance.

```
// ===== BEFORE: MySQL PDO Connection =====
$host = "localhost";
$dbname = "mydatabase";
$user = "root";
$pass = "";

$pdo = new PDO("mysql:host=$host;dbname=$dbname", $user, $pass);
$pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);

// ===== AFTER: Turso LibSQL Connection =====
use Turso\LibSQL\Client;

$url = getenv("TURSO_DB_URL");
$token = getenv("TURSO_AUTH_TOKEN");

$db = Client::withUrl($url, $token);
```

4.3 Before and After: Prepared Statements

Prepared statements are the primary defense against SQL injection. The Turso SDK supports parameterized queries with named parameters. The syntax differs from PDO but achieves the same result. Note that LibSQL uses positional parameters (?1, ?2) instead of PDO's named parameters.

```
// ===== BEFORE: MySQL PDO Prepared Statement =====
$stmt = $pdo->prepare("SELECT * FROM users WHERE email = :email");
$stmt->execute(["email" => $email]);
$user = $stmt->fetch(PDO::FETCH_ASSOC);

// ===== AFTER: Turso LibSQL Prepared Statement =====
$result = $db->execute("SELECT * FROM users WHERE email = ?1", [$email]);
$rows = $result->rows;
$user = $rows[0] ?? null;
```

4.4 Environment-Aware Configuration File

Create a centralized database configuration file that reads connection parameters from environment variables. This file will be included by every PHP script that needs database access. The configuration should automatically detect the environment (local vs production) and use the appropriate settings. Create this file at `config/database.php` in your project.

```
<?php // config/database.php

require_once __DIR__ . '/../vendor/autoload.php';
use Turso\LibSQL\Client;

function getDb(): Turso\LibSQL\Client {
    $url = getenv("TURSO_DB_URL");
    $token = getenv("TURSO_AUTH_TOKEN");

    if (!$url || !$token) {
        throw new RuntimeException("Missing Turso DB credentials");
    }

    return Client::withUrl($url, $token);
}

?>
```

4.5 Common Query Pattern Migrations

Beyond basic `SELECT` queries, you will need to convert `INSERT`, `UPDATE`, `DELETE`, and transaction operations. The Turso SDK handles all of these through the same `execute()` method. For transactions, use the `transaction()` method on the client. Here are the most common patterns you will encounter during the migration.

```
// INSERT
$db->execute("INSERT INTO users (name, email) VALUES (?, ?)", [
    $name, $email
]);
$lastId = $db->execute("SELECT last_insert_rowid()")->rows[0][0];

// UPDATE
$db->execute("UPDATE users SET name = ?1 WHERE id = ?2", [
    $name, $userId
]);

// DELETE
$db->execute("DELETE FROM users WHERE id = ?1", [$userId]);

// TRANSACTION
$db->transaction(function ($db) use ($data) {
    $db->execute("INSERT INTO orders (user_id, total) VALUES (?, ?)", [
        $data["user_id"], $data["total"]
```

```
});  
$db->execute("UPDATE inventory SET stock = stock - ?1 WHERE product_id = ?2", [  
$data["quantity"], $data["product_id"]  
]);  
});
```

Stage 5: Preparing PHP for Vercel

With the database layer refactored, the next step is to configure your project for Vercel's deployment model. This involves creating the `vercel.json` configuration file, organizing your public assets, and setting up routing rules that map URL patterns to the correct PHP files. Vercel treats each PHP file as a serverless function, so your application's routing architecture must be designed with this constraint in mind.

5.1 How Vercel Handles PHP

When Vercel deploys a PHP project, it uses the `@vercel/php` builder (specified in the build configuration). This builder scans your project for PHP files and creates a serverless function for each one. Files in the `api/` directory are automatically treated as API endpoints. Files in the `public/` directory are served as static assets (images, CSS, JavaScript). PHP files in `public/` are also executed but with different routing behavior.

The execution model is fundamentally different from traditional PHP hosting. On XAMPP, Apache receives a request for `/about.php` and executes it directly. On Vercel, the same request hits a serverless function that boots a PHP runtime, executes the file, and returns the response. This means each request has a cold-start penalty (typically 200-500ms for PHP on Vercel), and there is no shared state between requests. Environment variables, sessions, and temporary data must be handled externally.

5.2 Required `vercel.json` Configuration

The `vercel.json` file is the central configuration for your Vercel deployment. It defines the build settings, routing rules, and headers. The following configuration covers the most common patterns for a PHP application. Place this file in your project root directory.

```
{  
  "builds": [  
    { "src": "api/**/*.php", "use": "@vercel/php" },  
    { "src": "public/**/*.php", "use": "@vercel/php" }  
  ],  
  "routes": [  
    { "src": "/api/(.*)", "dest": "/api/$1" },  
    { "src": "/(.*)", "dest": "/public/$1" },  
  ]  
}
```

```

{ "handle": "filesystem" },
{ "src": "/(.*)", "dest": "/public/index.php" }
],
"headers": [
{
"source": "/api/(.*)",
"headers": [
{ "key": "Access-Control-Allow-Origin", "value": "*" },
{ "key": "Access-Control-Allow-Methods", "value": "GET,POST,PUT,DELETE,OPTIONS"
},
{ "key": "Access-Control-Allow-Headers", "value": "Content-Type,Authorization" }
]
}
]
}

```

5.3 Folder Structure for Vercel

Vercel expects a specific folder structure. The `api/` directory is reserved for serverless API endpoints. The `public/` directory contains static assets and serves as the document root. PHP files in `public/` are executed as serverless functions when accessed via their URL path. Shared include files and configuration should live outside both directories to prevent direct HTTP access.

```

project-root/
|-- api/ # API endpoints: /api/users.php
| |-- users.php # -> https://yourdomain.com/api/users
| |-- products.php
| |-- auth/
| |-- login.php
|-- public/ # Document root (static + executable)
| |-- index.php # -> https://yourdomain.com/
| |-- about.php # -> https://yourdomain.com/about
| |-- css/style.css # Static file
| |-- js/app.js # Static file
| |-- images/ # Static files
|-- config/ # NOT accessible via HTTP
|-- includes/ # NOT accessible via HTTP
|-- vendor/ # Composer dependencies
|-- vercel.json
|-- composer.json

```

5.4 Routing Adjustments

If your application uses direct `.php` file references in URLs (e.g., `/about.php`, `/contact.php`), these will continue to work if the files are in the `public/` directory. However, for cleaner URLs (e.g., `/about`, `/contact`), add rewrite rules to `vercel.json`. The routing system in Vercel processes rules in order, so more specific patterns should come before general catch-all patterns.

```
// Add these to "routes" array in vercel.json:
{ "src": "/about", "dest": "/public/about.php" },
{ "src": "/contact", "dest": "/public/contact.php" },
{ "src": "/login", "dest": "/public/login.php" },
{ "src": "/dashboard/(.*)", "dest": "/public/dashboard.php?page=$1" },
```

Stage 6: Deploying to Vercel

With your project prepared, configured, and pushed to GitHub, you are ready to deploy to Vercel. This stage covers the complete deployment process including GitHub integration, environment variable configuration, build settings, and production deployment. The entire process can be completed in under 15 minutes using the Vercel dashboard.

6.1 Vercel Dashboard Deployment Steps

Follow these steps to deploy your PHP project on Vercel. The process connects your GitHub repository to Vercel, which enables automatic deployments on every push to the main branch. This CI/CD integration is one of Vercel's strongest features and eliminates the need for manual deployment workflows.

Step-by-step Vercel deployment:
1. Go to https://vercel.com/new
2. Sign in with your GitHub account
3. Select [YOUR_GITHUB_REPO] from the repository list
4. Configure the project:
- Framework Preset: "Other"
- Root Directory: "." (project root)
- Build Command: (leave empty - auto-detected)
- Output Directory: (leave empty)
5. Click "Deploy"
6. Wait for build to complete (2-5 minutes for first build)
7. Vercel assigns a preview URL: https://yourproject.vercel.app

6.2 Environment Variable Setup

After the initial deployment, configure the environment variables that your application needs. Navigate to your project's Settings tab in the Vercel dashboard, select Environment Variables, and add the following variables. These variables are accessible in your PHP code via `getenv()` and are encrypted at rest.

Variable	Example Value	Description	Required
TURSO_DB_URL	libsql://mydb-myorg.turso.io	Turso database connection URL	Yes
TURSO_AUTH_TOKEN	eyJhbGciOiJIUzI1NiIsIn...	Turso authentication token	Yes
APP_ENV	production	Application environment	Yes
APP_URL	https://yourdomain.com	Base URL for dynamic links	Yes
APP_KEY	base64:random32chars...	Encryption key for sessions	Yes

Table 4: Required Vercel Environment Variables

NOTE: Environment variables can be scoped to specific environments (Production, Preview, Development). Use this feature to have different Turso databases for testing and production, which prevents accidental data corruption during development.

6.3 Custom Domain Setup

By default, Vercel provides a `.vercel.app` subdomain. For production use, add a custom domain. Navigate to Settings then Domains in the Vercel dashboard. Enter your domain name (e.g., `app.yourdomain.com`) and follow the DNS configuration instructions. Vercel automatically provisions an SSL certificate via Let's Encrypt, so HTTPS is handled without any additional configuration on your part.

For the DNS configuration, add a CNAME record pointing your subdomain to `cname.vercel-dns.com`. If you are using a root domain (e.g., `yourdomain.com`), use A records pointing to `76.76.21.21`. DNS propagation typically takes 5-30 minutes. Vercel will automatically issue the SSL certificate once DNS is verified.

Stage 7: Handling Hardcoded Localhost URLs and Paths

One of the most common issues when migrating from a local XAMPP setup to a production deployment is hardcoded localhost URLs and absolute Windows paths. These references break immediately when the application runs on Vercel. This stage shows you how to systematically find and replace all hardcoded references with environment-aware dynamic values.

7.1 Dynamic Base URL Configuration

Create a helper function that returns the correct base URL based on the current environment. This function should be used everywhere in your application instead of hardcoded URLs. It reads the APP_URL environment variable (set in Vercel) and falls back to localhost for local development. Create this file at config/app.php.

```
<?php // config/app.php

function base_url(string $path = ""): string {
    $baseUrl = getenv("APP_URL") ?: "http://localhost";
    return rtrim($baseUrl, "/") . "/" . ltrim($path, "/");
}

function asset_url(string $path): string {
    return base_url($path);
}

function redirect(string $path): void {
    header("Location: " . base_url($path));
    exit;
}

?>
```

7.2 Before and After: Common Hardcoded Patterns

Below are the most common hardcoded patterns found in XAMPP PHP projects and their corrected versions using the dynamic configuration. Search your entire codebase for these patterns and replace them systematically.

Pattern	Before (Hardcoded)	After (Dynamic)
Form action	<form action="http://localhost/project/contact.php">	<form action="<?= base_url("contact") ?>">
Link href		<a href="<?= base_url("about") ?>">
Image src		">
Include path	require "C:/xampp/htdocs/project/includes/header.php"	require __DIR__ . '/../includes/header.php"
API endpoint	fetch("http://localhost/project/api/users")	fetch(base_url + "/api/users")
Header redirect	header("Location: /project/dashboard.php")	redirect("dashboard")

Pattern	Before (Hardcoded)	After (Dynamic)
CSS link	<link href="css/style.css">	<link href="<?= asset_url("css/style.css") ?>">

Table 5: Hardcoded Pattern Replacement Guide

7.3 Bulk Search and Replace Script

Use this shell script to identify all files containing hardcoded localhost references. Run this from your project root. Review each file manually before making changes, as some references (like documentation or comments) may not need modification. The script generates a report file listing every occurrence with file path and line number.

```
# Search for hardcoded localhost URLs
echo "=== Hardcoded localhost URLs ===" > migration_audit.txt
grep -rn "localhost" --include="*.php" . >> migration_audit.txt
grep -rn "127\.0\.0\.1" --include="*.php" . >> migration_audit.txt

# Search for absolute Windows/XAMPP paths
echo "\n=== Absolute Windows paths ===" >> migration_audit.txt
grep -rn "C:/xampp" --include="*.php" . >> migration_audit.txt
grep -rn "C:\\xampp" --include="*.php" . >> migration_audit.txt

# Search for hardcoded .php extensions in URLs
echo "\n=== Hardcoded .php URL references ===" >> migration_audit.txt
grep -rn 'href=".*\.php"' --include="*.php" . >> migration_audit.txt
grep -rn 'action=".*\.php"' --include="*.php" . >> migration_audit.txt

# View the audit report
cat migration_audit.txt
```

Stage 8: SSL, Environment Security, and Production Hardening

Security is not an afterthought; it is a continuous process. This stage covers the security measures you must implement for your production deployment on Vercel. Vercel handles some security concerns automatically (SSL, DDoS protection), but application-level security remains your responsibility. The following checklist ensures your application meets production security standards.

8.1 HTTPS and SSL

Vercel automatically provisions SSL certificates via Let's Encrypt for all deployments, including custom domains. HTTPS is enforced by default and HTTP requests are automatically redirected to HTTPS. You do not need to configure anything for SSL. However, ensure that all hardcoded URLs in your

application use `https://` (handled by the `APP_URL` environment variable from Stage 7), and that mixed content warnings are avoided by using protocol-relative URLs or the dynamic base URL function.

8.2 Secure Headers

Add security headers to all responses. Vercel allows you to configure headers globally in the `vercel.json` file. The following headers provide protection against common web vulnerabilities including clickjacking, MIME-type sniffing, and cross-site scripting. Add these to the `headers` array in your `vercel.json`.

```
// Add to "headers" array in vercel.json:
{
  "source": "/*",
  "headers": [
    { "key": "X-Content-Type-Options", "value": "nosniff" },
    { "key": "X-Frame-Options", "value": "DENY" },
    { "key": "X-XSS-Protection", "value": "1; mode=block" },
    { "key": "Referrer-Policy", "value": "strict-origin-when-cross-origin" },
    { "key": "Permissions-Policy", "value": "camera=(), microphone=(), geolocation=()" }
  ]
}
```

8.3 Hiding PHP Errors in Production

PHP error messages contain sensitive information about your application's file structure, database queries, and server configuration. In production, all errors should be logged to a file or external service rather than displayed to users. Configure error reporting at the top of your main entry point file (e.g., `public/index.php`) or in a shared header file.

```
<?php // Add at top of public/index.php or includes/headers.php
// Disable error display in production
if (getenv("APP_ENV") === "production") {
  ini_set("display_errors", "0");
  ini_set("log_errors", "1");
  error_reporting(E_ALL);
  // Log to stderr (captured by Vercel)
  ini_set("error_log", "php://stderr");
} else {
  ini_set("display_errors", "1");
  error_reporting(E_ALL);
}
```

?>

8.4 Secrets Management

Never commit secrets (API keys, database tokens, encryption keys) to your Git repository. Vercel's environment variables are encrypted at rest and injected at build time, making them the recommended way to store secrets. For additional security, use Vercel's Edge Config for secrets that need to change without redeployment. Rotate your Turso auth token periodically (every 90 days is recommended) and update the environment variable in the Vercel dashboard.

WARNING: If you accidentally commit secrets to GitHub, they remain in your Git history even after deletion. Use the GitHub Secret Scanning feature or tools like `git-filter-repo` to remove sensitive data from your repository history. Rotate all compromised credentials immediately.

Stage 9: Post-Deployment Testing

After deploying to Vercel, you must verify that every feature of your application works correctly in the production environment. Testing a serverless PHP application requires a different approach than testing a traditional XAMPP setup, because each request is independent, the filesystem is ephemeral, and environment variables are the sole configuration mechanism.

9.1 Testing Checklist

Work through this checklist systematically after each deployment. Each item tests a specific aspect of your application's functionality in the Vercel serverless environment. Check every item and document any failures for debugging.

#	Test Case	Expected Result	How to Test
1	Homepage loads	200 OK, full HTML rendered	Visit root URL in browser
2	All pages load	200 OK for each page	Test every .php page link
3	API endpoints respond	JSON response, correct data	Use curl or Postman
4	Database reads work	Correct data from Turso	Browse data-heavy pages
5	Database writes work	Data persisted to Turso	Submit a form, verify in DB
6	Environment variables	Config values loaded	Add <code>?debug=1</code> to URL, print env
7	HTTPS redirect	HTTP redirects to HTTPS	Visit <code>http://</code> version

#	Test Case	Expected Result	How to Test
8	Static assets load	CSS/JS/images 200 OK	Check browser Network tab
9	Custom domain resolves	SSL cert valid, page loads	Visit custom domain
10	Error pages display	Custom 404/500 pages	Visit non-existent URL
11	Cold start latency	Page loads in under 3s	Clear cache, test first load
12	CORS headers present	API responds to cross-origin	Test from different origin

Table 6: Post-Deployment Testing Checklist

9.2 Database Connectivity Check

Create a temporary diagnostic endpoint to verify Turso connectivity from Vercel. This file should be removed after testing. It attempts to connect to Turso, run a simple query, and returns the result as JSON. If this returns an error, your Turso credentials or network configuration are incorrect.

```
<?php // api/health.php (REMOVE after testing)
header("Content-Type: application/json");
require_once __DIR__ . "/../config/database.php";
try {
    $db = getDb();
    $result = $db->execute("SELECT 1 as test");
    echo json_encode(["status" => "ok", "db" => "connected"]);
} catch (Exception $e) {
    http_response_code(500);
    echo json_encode(["status" => "error", "message" => $e->getMessage()]);
}
?>
```

9.3 Debugging Strategy

When something goes wrong in production, use Vercel's built-in debugging tools. The Vercel dashboard provides real-time logs for every deployment. Navigate to your project's Logs tab to see output from `error_log()`, `echo`, and `var_dump()` statements. For more detailed debugging, use the Vercel CLI (`npm i -g vercel`) and run `vercel logs [deployment-url]` to stream logs in real-time from your terminal.

Common issues to check: missing environment variables (the most common cause of 500 errors), Turso auth token expiration or invalidation, incompatible SQL syntax that works in MySQL but fails in

LibSQL, and cold start timeouts for complex PHP files that include many dependencies. For cold start issues, consider combining multiple PHP files into fewer, larger serverless functions to reduce the number of cold starts per page load.

9.4 Performance Checks

Monitor your application's performance after deployment using Vercel's Analytics (available on Pro plans) or external tools like Google PageSpeed Insights, GTmetrix, and WebPageTest. Key metrics to watch include Time to First Byte (TTFB), which should be under 500ms for serverless PHP; First Contentful Paint (FCP), which should be under 1.8 seconds; and Total Blocking Time (TBT), which should be under 200ms. If TTFB is consistently above 1 second, consider upgrading your Vercel plan (which provides more memory and CPU for serverless functions) or optimizing your PHP code to reduce bootstrap time.

Stage 10: Troubleshooting Reference

This table covers the most common errors encountered during PHP to Vercel migration with Turso, their root causes, and step-by-step fixes. Use this as your first reference when something goes wrong. Each entry has been compiled from real-world migration scenarios.

Error	Likely Cause	Fix
500 Internal Server Error on all pages	Missing TURSO_DB_URL or TURSO_AUTH_TOKEN env var	Add both variables in Vercel Settings then redeploy
500 Error on specific PHP page	PHP syntax error or fatal exception in that file	Check Vercel Logs for stack trace; fix the error and push
Turso "authentication failed"	Invalid or expired auth token	Generate new token: <code>turso db tokens create [DB_NAME]</code>
Turso "database not found"	Wrong database URL or database was deleted	Verify URL with: <code>turso db show [DB_NAME] --url</code>
Page not found (404)	File not in public/ or route not in vercel.json	Move file to public/ or add route rewrite rule
Static assets return 404	Assets not in public/ directory	Move CSS/JS/images to public/ directory
PHP session not persisting	Vercel filesystem is ephemeral	Use Upstash Redis or external session storage
File upload fails	No persistent filesystem on Vercel	Use S3, Cloudflare R2, or Uploadthing for storage

Error	Likely Cause	Fix
SQL syntax error	MySQL-specific syntax in LibSQL query	Check Table 3 for syntax differences; rewrite query
Cold start too slow (3s+)	Large PHP bootstrap (many includes)	Combine files, use autoloader, reduce dependencies
CORS error on API call	Missing CORS headers in vercel.json	Add Access-Control headers in vercel.json headers array
Mixed content warning	HTTP resource loaded on HTTPS page	Use asset_url() helper for all asset references
composer install fails on Vercel	Missing or invalid composer.json	Verify composer.json is valid; test locally first
AUTO_INCREMENT error	MySQL syntax used in LibSQL CREATE TABLE	Change to: id INTEGER PRIMARY KEY AUTOINCREMENT
Environment variable is null	Variable not set in Vercel dashboard	Add in Settings then redeploy (env vars require redeploy)

Table 7: Comprehensive Troubleshooting Reference

Appendix A: Recommended Architecture for Long-Term Scalability

If you plan to scale your application beyond a simple PHP website, consider migrating to a modern architecture that separates concerns and leverages the strengths of each platform component. The recommended long-term architecture follows a decoupled frontend-backend pattern where the frontend is a static site or Single Page Application (SPA) deployed on Vercel's Edge Network, and the backend is a set of serverless API functions also on Vercel, communicating with Turso for data persistence.

This architecture provides several advantages over a monolithic PHP application. First, the frontend can be cached at the edge, resulting in sub-100ms page loads globally. Second, the API layer can scale independently based on demand. Third, the Turso database handles read replicas automatically, reducing latency for users in different geographic regions. Fourth, the separation of concerns makes the codebase easier to maintain, test, and extend over time.

For a gradual migration path, start by converting your PHP pages to API endpoints that return JSON data. Then build a lightweight frontend (using plain HTML/JS or a framework like Alpine.js or htmx for minimal complexity) that consumes those API endpoints. This approach lets you migrate incrementally without rewriting the entire application at once. Each page can be migrated independently: convert the backend to an API, then update the frontend to consume it, then move to the next page.

Appendix B: Platform Recommendation

Based on your current setup (raw PHP website on XAMPP with MySQL), here is the honest recommendation. The right choice depends on your project's specific requirements, your team's technical expertise, and your budget constraints. Each option below includes a clear assessment of when it is the best fit.

Option	When to Choose	Pros	Cons
Stay: PHP on Vercel + Turso	API-heavy app, static frontend, edge deployment needed	Free tier, global CDN, automatic SSL, Git integration	No persistent filesystem, cold starts, PHP community runtime only
Move: Laravel on Railway	Full-stack PHP app, sessions, uploads, background jobs	Native PHP support, persistent FS, managed DB, cron jobs	No edge deployment, \$5+/month, smaller free tier
Use: Traditional Hosting (Hostinger)	Simple PHP website, no scaling needs, budget-conscious	Cheapest option, cPanel, native PHP, email hosting included	No CI/CD, no edge, manual deployments, limited scaling
Hybrid: Frontend on Vercel + Backend on Railway	Complex app needing both edge frontend and persistent backend	Best of both worlds, scalable, modern tooling	More complex setup, two platforms to manage, higher cost

Table 8: Platform Recommendation Matrix

Appendix C: Migration Difficulty Estimate

Based on the complexity analysis of all 10 stages, the overall migration difficulty is rated as **Medium**. Here is the breakdown by stage, with estimated time for a developer familiar with PHP and basic Git usage but new to Vercel and Turso. Total estimated time is 2-4 days for a small-to-medium PHP project (under 20 pages, under 10 database tables).

Stage	Description	Difficulty	Est. Time
1	Audit and Compatibility Check	Easy	1-2 hours
2	GitHub Preparation	Easy	30-60 minutes
3	Migrating MySQL to Turso	Medium-Hard	3-5 hours
4	Refactoring PHP Database Layer	Hard	4-8 hours

Stage	Description	Difficulty	Est. Time
5	Preparing PHP for Vercel	Medium	1-2 hours
6	Deploying to Vercel	Easy	30-60 minutes
7	Handling Hardcoded URLs/Paths	Easy-Medium	1-3 hours
8	SSL and Security Hardening	Easy	1-2 hours
9	Post-Deployment Testing	Easy	2-3 hours
10	Troubleshooting	Ongoing	Varies

Table 9: Stage-by-Stage Difficulty and Time Estimate

The hardest stages are 3 (database migration) and 4 (PHP database refactoring), which together account for roughly 60% of the total effort. Stages 1, 2, 6, 8, and 9 are straightforward and can be completed quickly. Stage 7 (hardcoded URLs) varies widely depending on code quality: a well-structured project may need only 30 minutes, while a legacy codebase with hundreds of hardcoded references could take several hours. If your project uses a framework (even a lightweight one) rather than raw procedural PHP, the migration will be significantly easier because configuration is typically centralized.

WARNING: This guide assumes a small-to-medium PHP project. For large applications (50+ pages, complex database schemas, session-heavy workflows, or heavy use of MySQL-specific features), the difficulty increases to Hard and the estimated time doubles to 5-10 days. Consider phasing the migration over multiple iterations rather than attempting a big-bang approach.